Matrix-Chain Multiplication

Problem: given a sequence $\langle A_1, A_2, \ldots, A_n \rangle$, compute the product:

$$A_1 \cdot A_2 \cdots A_n$$

- Matrix compatibility:

$C = A \cdot B$

$col_A = row_B$

$row_C = row_A$

$col_C = col_B$

$C = A_1 \cdot A_2 \cdots A_i \cdot A_{i+1} \cdots A_n$

$col_i = row_{i+1}$

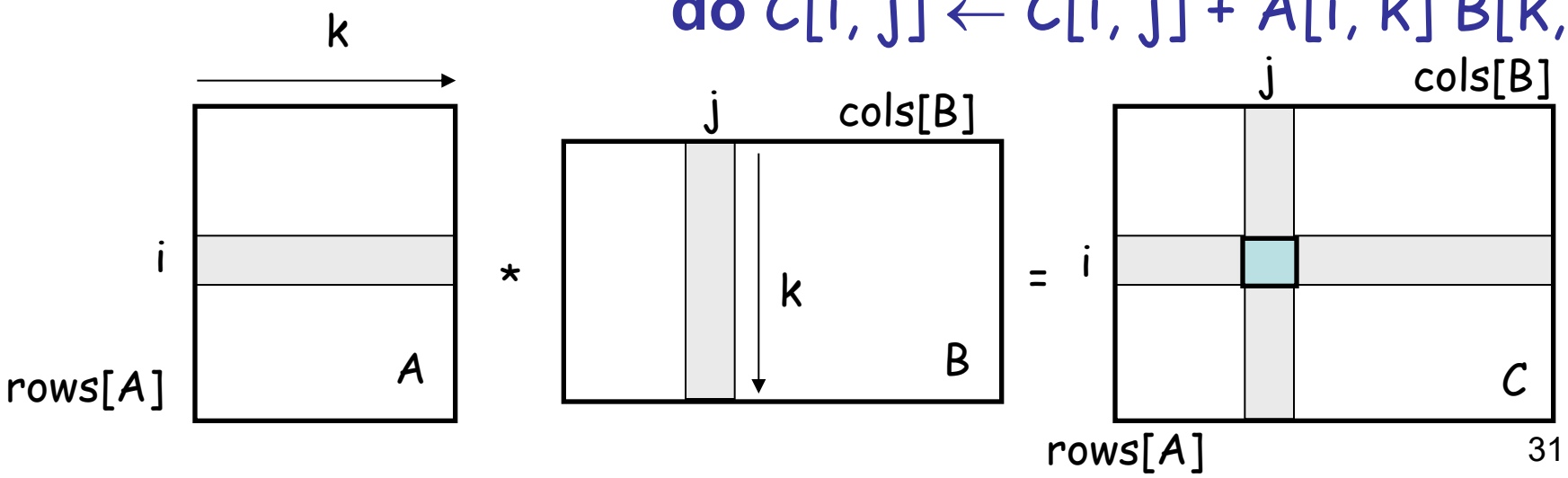
$row_C = row_{A1}$

$col_C = col_{An}$

MATRIX-MULTIPLY(A, B)

```
if columns[A] ≠ rows[B]
   then error "incompatible dimensions"
   else for i ← 1 to rows[A]
            do for j ← 1 to columns[B]
                   do C[i, j] = 0
                      for k ← 1 to columns[A]
                          do C[i, j] ← C[i, j] + A[i, k] B[k, j]
```

rows[A] · cols[A] · cols[B] multiplications



Matrix-Chain Multiplication

- In what order should we multiply the matrices?

$$A_1 \cdot A_2 \cdots A_n$$

- Parenthesize the product to get the order in which matrices are multiplied
- *E.g.:* $A_1 \cdot A_2 \cdot A_3 = ((A_1 \cdot A_2) \cdot A_3)$

$$= (A_1 \cdot (A_2 \cdot A_3))$$

- Which one of these orderings should we choose?
  - The order in which we multiply the matrices has a significant impact on the cost of evaluating the product

Example

$$A_1 \cdot A_2 \cdot A_3$$

- A_1: 10 x 100
- A_2: 100 x 5
- A_3: 5 x 50

1. $((A_1 \cdot A_2) \cdot A_3)$: $A_1 \cdot A_2 = 10 \times 100 \times 5 = 5{,}000$ (10 x 5)

   $((A_1 \cdot A_2) \cdot A_3) = 10 \times 5 \times 50 = 2{,}500$

   Total: 7,500 scalar multiplications

2. $(A_1 \cdot (A_2 \cdot A_3))$: $A_2 \cdot A_3 = 100 \times 5 \times 50 = 25{,}000$ (100 x 50)

   $(A_1 \cdot (A_2 \cdot A_3)) = 10 \times 100 \times 50 = 50{,}000$

   Total: 75,000 scalar multiplications

one order of magnitude difference!!

Matrix-Chain Multiplication: Problem Statement

- Given a chain of matrices $\langle A_1, A_2, \ldots, A_n \rangle$, where A_i has dimensions $p_{i-1} \times p_i$, fully parenthesize the product $A_1 \cdot A_2 \cdots A_n$ in a way that minimizes the number of scalar multiplications.

$$\begin{array}{ccccccccc} A_1 & \cdot & A_2 & \cdots & A_i & \cdot & A_{i+1} & \cdots & A_n \\ p_0 \times p_1 & & p_1 \times p_2 & & p_{i-1} \times p_i & & p_i \times p_{i+1} & & p_{n-1} \times p_n \end{array}$$

What is the number of possible parenthesizations?

- Exhaustively checking all possible parenthesizations is not efficient!
- It can be shown that the number of parenthesizations grows as $\Omega(4^n/n^{3/2})$

  (see page 333 in your textbook)

1. The Structure of an Optimal Parenthesization

- Notation:

$$A_{i\ldots j} = A_i\, A_{i+1} \cdots A_j,\ i \le j$$

- Suppose that an optimal parenthesization of $A_{i\ldots j}$ splits the product between A_k and A_{k+1}, where $i \le k < j$

$$\begin{aligned} A_{i\ldots j} &= A_i\, A_{i+1} \cdots A_j \\ &= A_i\, A_{i+1} \cdots A_k\, A_{k+1} \cdots A_j \\ &= A_{i\ldots k}\, A_{k+1\ldots j} \end{aligned}$$

Optimal Substructure

$$A_{i\ldots j} = A_{i\ldots k}\, A_{k+1\ldots j}$$

- The parenthesization of the "prefix" $A_{i\ldots k}$ must be an optimal parentesization
- If there were a less costly way to parenthesize $A_{i\ldots k}$, we could substitute that one in the parenthesization of $A_{i\ldots j}$ and produce a parenthesization with a lower cost than the optimum $\Rightarrow$ contradiction!
- An optimal solution to an instance of the matrix-chain multiplication contains within it optimal solutions to subproblems

2. A Recursive Solution

- Subproblem:

  determine the minimum cost of parenthesizing

  $$A_{i \ldots j} = A_i A_{i+1} \cdots A_j \qquad \text{for } 1 \le i \le j \le n$$

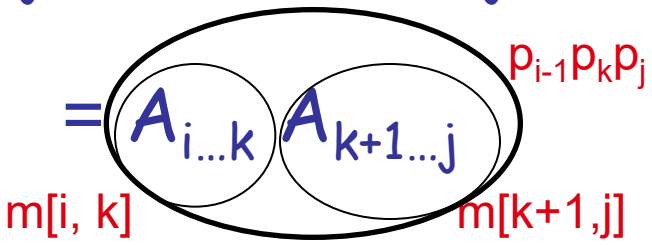
- Let $m[i, j]$ = the minimum number of multiplications needed to compute $A_{i \ldots j}$
  - full problem ($A_{1..n}$): $m[1, n]$
  - $i = j$: $A_{i \ldots i} = A_i \Rightarrow m[i, i] = 0$, for $i = 1, 2, \ldots, n$

2. A Recursive Solution

- Consider the subproblem of parenthesizing

$$A_{i \ldots j} = A_i A_{i+1} \cdots A_j \qquad \text{for } 1 \le i \le j \le n$$

$$= A_{i \ldots k} \; A_{k+1 \ldots j} \qquad \text{for } i \le k < j$$



- Assume that the optimal parenthesization splits the product $A_i A_{i+1} \cdots A_j$ at k ($i \le k < j$)

$$m[i, j] = \underbrace{m[i, k]}_{\text{min \# of multiplications to compute } A_{i \ldots k}} + \underbrace{m[k+1, j]}_{\text{min \# of multiplications to compute } A_{k+1 \ldots j}} + \underbrace{p_{i-1}p_kp_j}_{\text{\# of multiplications to compute } A_{i \ldots k}A_{k \ldots j}}$$

2. A Recursive Solution (cont.)

$$m[i, j] = m[i, k] + m[k+1, j] + p_{i-1} p_k p_j$$

- We do not know the value of k
  - There are j – i possible values for k: k = i, i+1, …, j-1
- Minimizing the cost of parenthesizing the product $A_i A_{i+1} \cdots A_j$ becomes:

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min\limits_{i \le k < j} \{m[i, k] + m[k+1, j] + p_{i-1} p_k p_j\} & \text{if } i < j \end{cases}$$

3. Computing the Optimal Costs

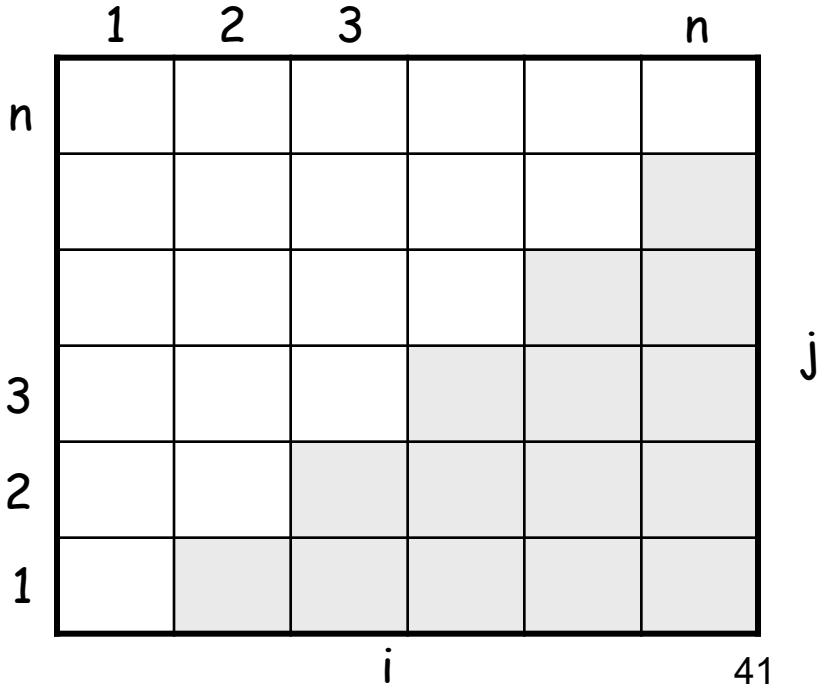
$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \le k < j} \{m[i, k] + m[k+1, j] + p_{i-1} p_k p_j\} & \text{if } i < j \end{cases}$$

- Computing the optimal solution recursively takes exponential time!
- How many subproblems?

  $\Rightarrow \Theta(n^2)$

  – Parenthesize $A_{i \ldots j}$ for $1 \le i \le j \le n$

  – One problem for each choice of i and j



3. Computing the Optimal Costs (cont.)

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min\limits_{i \le k < j} \{m[i, k] + m[k+1, j] + p_{i-1} p_k p_j\} & \text{if } i < j \end{cases}$$

- How do we fill in the tables m[1..n, 1..n]?
  - Determine which entries of the table are used in computing m[i, j]

$$A_{i \ldots j} = A_{i \ldots k} \, A_{k+1 \ldots j}$$

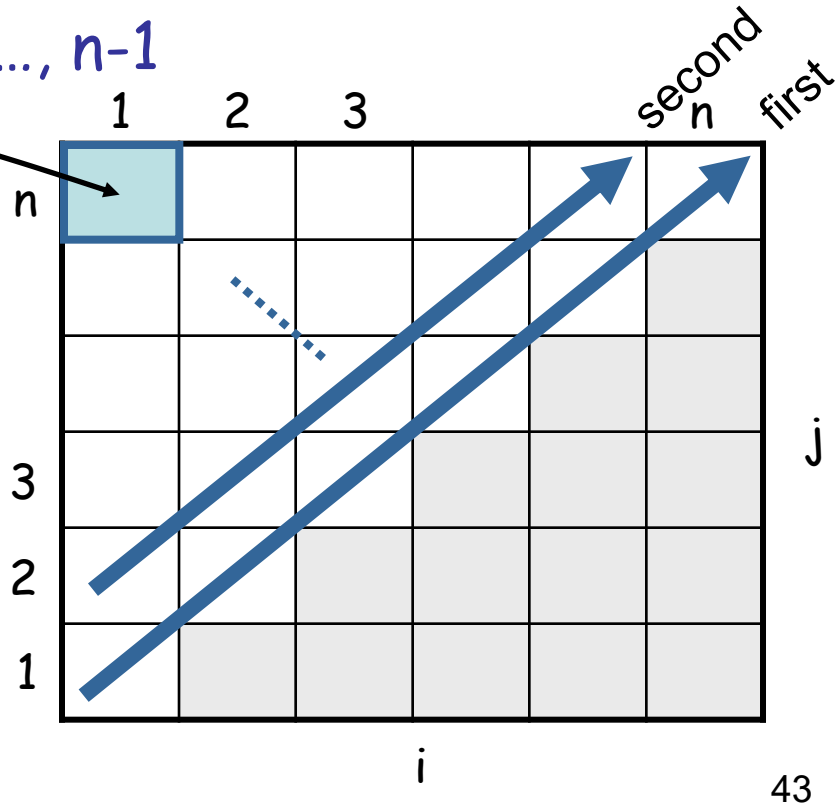
  - Subproblems' size is one less than the original size
  - **Idea:** fill in m such that it corresponds to solving problems of increasing length

3. Computing the Optimal Costs (cont.)

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min\limits_{i \le k < j} \{m[i, k] + m[k+1, j] + p_{i-1} p_k p_j\} & \text{if } i < j \end{cases}$$

- Length = 1: $i = j$, $i = 1, 2, \ldots, n$
- Length = 2: $j = i + 1$, $i = 1, 2, \ldots, n-1$

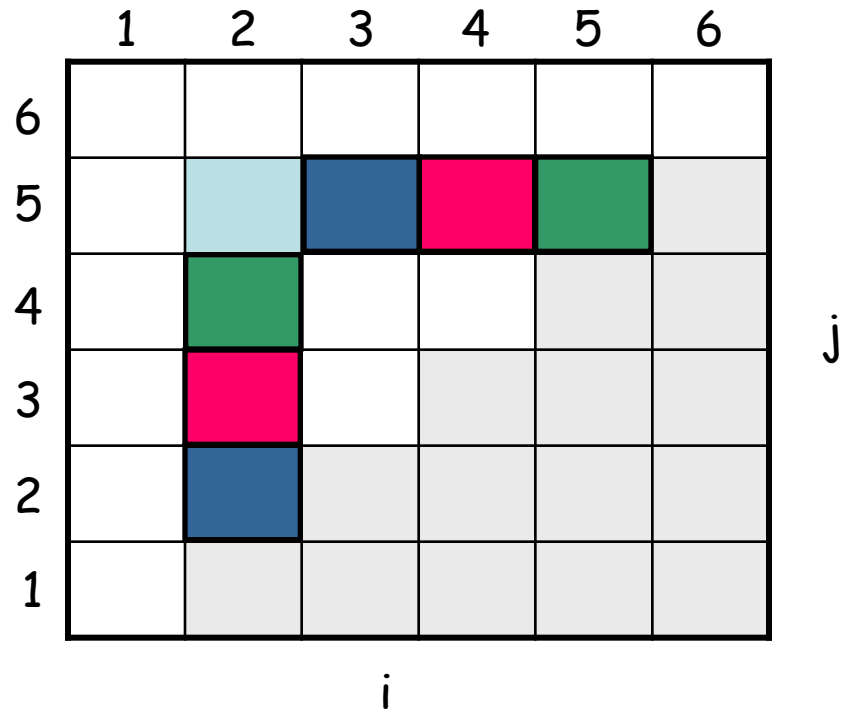
$m[1, n]$ gives the optimal solution to the problem



Compute rows from bottom to top and from left to right

Example: $\min \{m[i, k] + m[k+1, j] + p_{i-1}p_k p_j\}$

$$m[2, 5] = \min \begin{cases} m[2, 2] + m[3, 5] + p_1p_2p_5 & k = 2 \\ m[2, 3] + m[4, 5] + p_1p_3p_5 & k = 3 \\ m[2, 4] + m[5, 5] + p_1p_4p_5 & k = 4 \end{cases}$$



- Values m[i, j] depend only on values that have been previously computed

Example $\min \{m[i, k] + m[k+1, j] + p_{i-1}p_k p_j\}$

Compute $A_1 \cdot A_2 \cdot A_3$

- A_1: 10 x 100 (p_0 x p_1)
- A_2: 100 x 5 (p_1 x p_2)
- A_3: 5 x 50 (p_2 x p_3)

	1	2	3
3	2 7500	2 25000	0
2	1 5000	0	
1	0		

$m[i, i] = 0$ for $i = 1, 2, 3$

$m[1, 2] = m[1, 1] + m[2, 2] + p_0p_1p_2$ (A_1A_2)

$= 0 + 0 + 10 * 100 * 5 = 5{,}000$

$m[2, 3] = m[2, 2] + m[3, 3] + p_1p_2p_3$ (A_2A_3)

$= 0 + 0 + 100 * 5 * 50 = 25{,}000$

$$m[1, 3] = \min \begin{cases} m[1, 1] + m[2, 3] + p_0p_1p_3 = 75{,}000 & (A_1(A_2A_3)) \\ m[1, 2] + m[3, 3] + p_0p_2p_3 = 7{,}500 & ((A_1A_2)A_3) \end{cases}$$

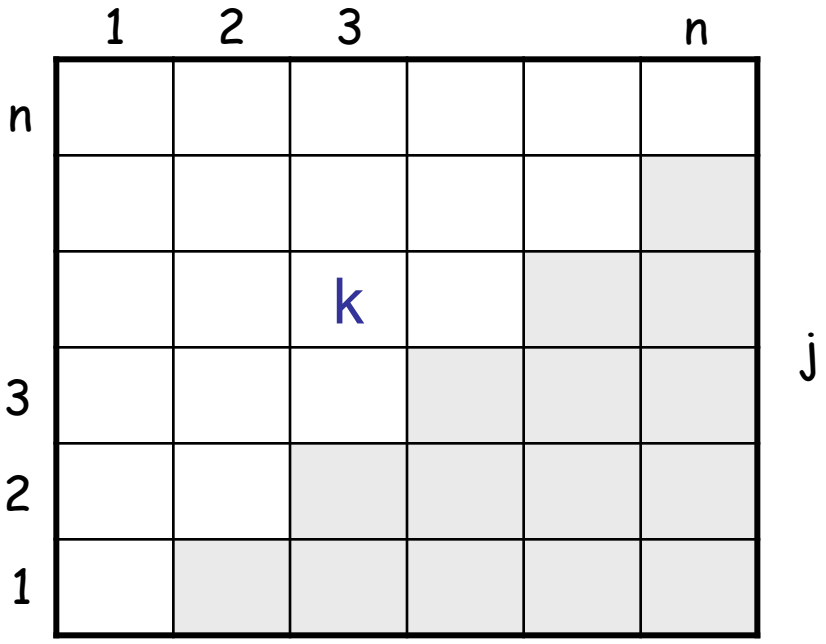
Matrix-Chain-Order(p)

```
MATRIX-CHAIN-ORDER(p)
 1  n ← length[p] − 1
 2  for i ← 1 to n
 3       do m[i, i] ← 0
 4  for l ← 2 to n                ▷ l is the chain length.
 5       do for i ← 1 to n − l + 1
 6            do j ← i + l − 1
 7               m[i, j] ← ∞
 8               for k ← i to j − 1
 9                    do q ← m[i, k] + m[k + 1, j] + p_{i−1} p_k p_j
10                       if q < m[i, j]
11                         then m[i, j] ← q
12                              s[i, j] ← k
13  return m and s
```

$O(N^3)$

4. Construct the Optimal Solution

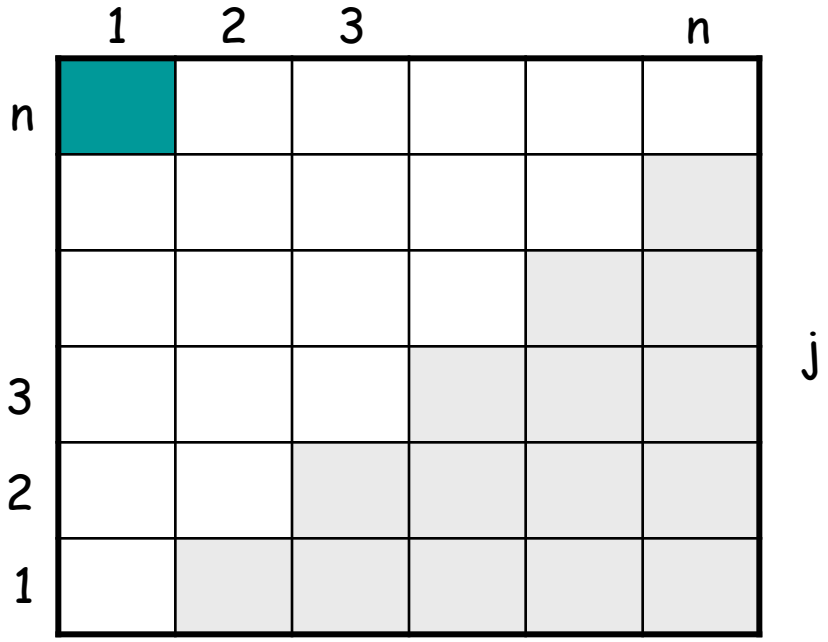
- In a similar matrix s we keep the optimal values of k
- $s[i, j]$ = a value of k such that an optimal parenthesization of $A_{i..j}$ splits the product between A_k and A_{k+1}



4. Construct the Optimal Solution

- $s[1, n]$ is associated with the entire product $A_{1..n}$
  - The final matrix multiplication will be split at k = $s[1, n]$

    $A_{1..n} = A_{1..s[1, n]} \cdot A_{s[1, n]+1..n}$
  - For each subproduct recursively find the corresponding value of k that results in an optimal parenthesization



4. Construct the Optimal Solution

- $s[i, j]$ = value of k such that the optimal parenthesization of $A_i A_{i+1} \cdots A_j$ splits the product between A_k and A_{k+1}

j \ i	1	2	3	4	5	6
6	3	3	3	5	5	-
5	3	3	3	4	-	
4	3	3	3	-		
3	1	2	-			
2	1	-				
1	-					

- $s[1, n] = 3 \Rightarrow A_{1..6} = A_{1..3} A_{4..6}$
- $s[1, 3] = 1 \Rightarrow A_{1..3} = A_{1..1} A_{2..3}$
- $s[4, 6] = 5 \Rightarrow A_{4..6} = A_{4..5} A_{6..6}$

4. Construct the Optimal Solution (cont.)

```
PRINT-OPT-PARENS(s, i, j)
if i = j
   then print "A"_i
   else print "("
        PRINT-OPT-PARENS(s, i, s[i, j])
        PRINT-OPT-PARENS(s, s[i, j] + 1, j)
        print ")"
```

	1	2	3	4	5	6
6	3	3	3	5	5	-
5	3	3	3	4	-	
4	3	3	3	-		
3	1	2	-			
2	1	-				
1	-					

i

j

Example: $A_1 \cdot \cdot \cdot A_6$ ((A_1 (A_2 A_3)) ((A_4 A_5) A_6))

```
PRINT-OPT-PARENS(s, i, j)
if i = j
   then print "A"_i
   else print "("
        PRINT-OPT-PARENS(s, i, s[i, j])
        PRINT-OPT-PARENS(s, s[i, j] + 1, j)
        print ")"
```

s[1..6, 1..6]

	1	2	3	4	5	6
6	3	3	3	5	5	-
5	3	3	3	4	-	
4	3	3	3	-		
3	1	2	-			
2	1	-				
1	-					

j

i

P-O-P(s, 1, 6) s[1, 6] = 3

i = 1, j = 6 "(" P-O-P (s, 1, 3) s[1, 3] = 1

i = 1, j = 3 "(" P-O-P(s, 1, 1) ⇒ "A_1"

P-O-P(s, 2, 3) s[2, 3] = 2

i = 2, j = 3 "(" P-O-P (s, 2, 2) ⇒ "A_2"

P-O-P (s, 3, 3) ⇒ "A_3"

")"

")" ...

- Example of Matrix chain Multiplication.
- Given
- **A_1 30×1**
- **A_2 1×40**
- **A_3 40×10**
- **A_4 10×25**

chains of length 1

i \ j	1	2	3	4
1	**0**			
2		**0**		
3			**0**	
4				**0**

A_1	30×1
A_2	1×40
A_3	40×10
A_4	10×25

chains of length 2

i \ j	1	2	3	4
1	0	1200 1		
2		0	400 2	
3			0	10000 3
4				0

A_1	30×1
A_2	1×40
A_3	40×10
A_4	10×25

$$m[1,2] = m[1,1] + m[2,2] + 30 \cdot 1 \cdot 40 = 1200$$

$$m[2,3] = m[2,2] + m[3,3] + 1 \cdot 40 \cdot 10 = 400$$

$$m[3,4] = m[3,3] + m[4,4] + 40 \cdot 10 \cdot 25 = 10000$$

chains of length 3

i \ j	1	2	3	4
1	0	1200 1	**700** **1**	
2		0	400 2	**650** **3**
3			0	10000 3
4				0

A_1	30×1
A_2	1×40
A_3	40×10
A_4	10×25

$$m[1,3] = \min \begin{cases} m[1,1] + m[2,3] + 30 \cdot 1 \cdot 10 = 700 \\ m[1,2] + m[3,3] + 30 \cdot 40 \cdot 10 = 13200 \end{cases} = 700$$

$$m[2,4] = \min \begin{cases} m[2,2] + m[3,4] + 1 \cdot 40 \cdot 25 = 11000 \\ m[2,3] + m[4,4] + 1 \cdot 10 \cdot 25 = 650 \end{cases} = 650$$

chains of length 4

i \ j	1	2	3	4
1	0	1200 1	700 1	**1400** **1**
2		0	400 2	650 3
3			0	10000 3
4				0

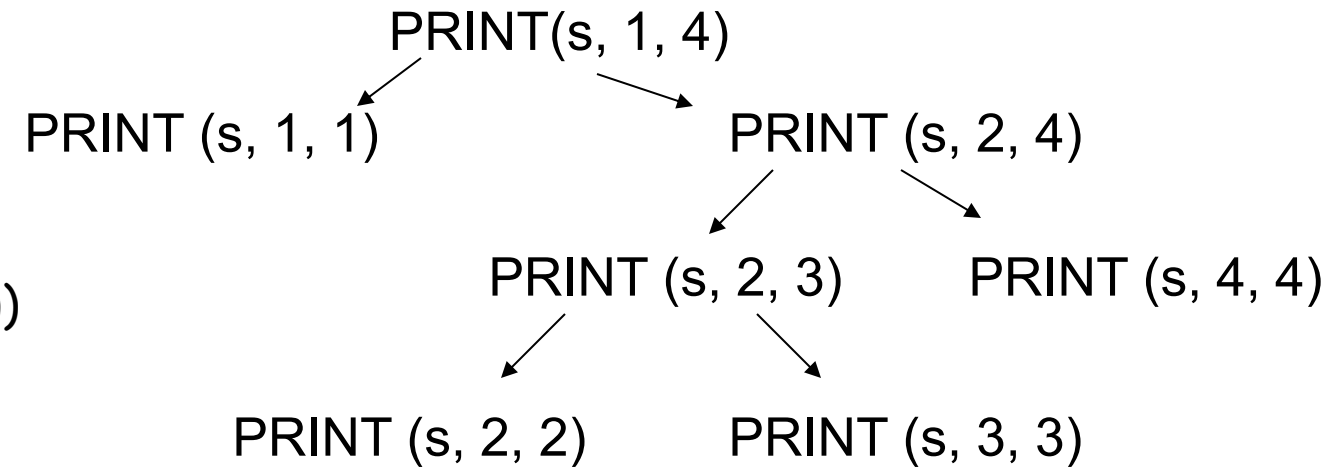
A_1	30×1
A_2	1×40
A_3	40×10
A_4	10×25

$$m[1,4] = \min \begin{cases} m[1,1] + m[2,4] + 30 \cdot 1 \cdot 25 = 1400 \\ m[1,2] + m[3,4] + 30 \cdot 40 \cdot 25 = 41200 = 1400 \\ m[1,3] + m[4,4] + 30 \cdot 10 \cdot 25 = 8200 \end{cases}$$

Printing the solution

i \ j	2	3	4
1	1	1	1
2		2	3
3			3

A_1	30×1
A_2	1×40
A_3	40×10
A_4	10×25



Output: $(A_1((A_2A_3)A_4))$